

AgriConnect Performance & Stress Test Report

Backend Stress Verification on Render Cloud Platform (50 Concurrent Users, 300s Duration)

Target Host Endpoint	https://e22-co2060-crop-yield-optimization-and.onrender.com
Execution Timestamp	September 25, 2026, 12:59 PM - 1:04 PM IST (5m 01s total)
Load Profile & Concurrency	50 Virtual Users (Threads) 60s Linear Ramp-Up 500ms Pacing Timer
Primary Result Status	PASSED — 100.00% Success Rate (8,760 Requests, 0 Errors)

1. Executive Summary

A formal stress and stability assessment was conducted against the AgriConnect backend deployed on Render cloud infrastructure. Over a 5-minute continuous run, 50 virtual users simulated representative farmer workflows, including initial authentication, farm queries, harvest yield aggregations, and support inquiry submissions. The backend proved completely resilient to service disruption, completing all 8,760 HTTP transactions with zero dropped connections or unhandled exceptions.

 **KEY VERDICT:** The system achieved a flawless 100% success rate with 0 errors across 8,760 transactions under 50 concurrent virtual users. Render did not experience container restarts, OOM kills, or 502/504 gateway timeouts. However, high CPU contention during concurrent bcrypt hashing caused an initial 18.8s login spike, lowering the overall APDEX index to 0.486.

TOTAL REQUESTS	ERROR RATE	AVG THROUGHPUT	AVG LATENCY
8,760	0.00%	29.12 /s	1,024 ms
100% Success Rate	0 HTTP Failures	Sustained Requests/sec	95th Pct: 1,611 ms

2. Transaction Performance & Response Times

The statistics table below details the performance profile of each discrete sampler executed during the run. All protected endpoints operated behind JWT verification.

Endpoint / Sampler	Count	Err %	Avg (ms)	Median	90% Pct	95% Pct	99% Pct	Max (ms)	Throughput
POST - Login (/api/users/login)	50	0.00%	2,650.92	1,680.50	5,183.00	8,918.75	18,830.00	18,830	0.82/s
GET - All Farms (/api/farms)	2,920	0.00%	1,194.04	1,115.50	1,481.90	1,692.95	2,273.23	7,952	10.23/s
GET - Harvest History (/api/farms/harvests)	2,901	0.00%	1,211.29	1,130.00	1,545.60	1,755.80	2,390.96	4,989	10.22/s
POST - Submit Inquiry (/api/inquiries)	2,889	0.00%	638.18	583.00	854.00	1,002.00	1,431.10	3,810	10.29/s
Total / Aggregate Performance	8,760	0.00%	1,024.09	1,010.00	1,415.90	1,611.85	2,218.68	18,830	29.12/s

3. APDEX (Application Performance Index) Analysis

APDEX quantifies end-user satisfaction by measuring response times against a target satisfaction threshold (T = 500 ms) and a toleration threshold (F = 1,500 ms). Requests under 500 ms are considered Satisfied; requests between 500 ms and 1,500 ms are Tolerated; and requests exceeding 1,500 ms or failing are classified as Frustrated.

Transaction Label	APDEX Score	T (Satisfied)	F (Tolerated)	User Experience Rating
POST - Login (/api/users/login)	0.220	500 ms	1,500 ms	Unacceptable (Heavy CPU queuing)
GET - Harvest History (/api/farms/harvests)	0.443	500 ms	1,500 ms	Poor / Tolerable
GET - All Farms (/api/farms)	0.455	500 ms	1,500 ms	Poor / Tolerable
POST - Submit Inquiry (/api/inquiries)	0.585	500 ms	1,500 ms	Fair / Tolerable

4. Deep-Dive Root Cause & Bottleneck Analysis

1. Login Latency Spike (18.83s Max Response Time)

During the first 60 seconds of test ramp-up, all 50 virtual users initiated login calls almost simultaneously. The authentication controller invokes ``bcrypt.compare(password, user.password)``. Bcrypt is an intentionally expensive, CPU-bound algorithm configured to execute multiple hashing rounds on the libuv thread pool. Because Render's container operates on a shared/burstable CPU environment with 512 MB memory, concurrent bcrypt calculations completely saturated the single virtual CPU core. This caused an HTTP connection queue backlog, inflating the 95th percentile login latency to 8.91s and maximum latency to 18.83s. Once all users authenticated and entered the operational loop, server latency stabilized immediately to ~1,000 ms.

2. Read Query vs. Write Query Discrepancy

Counterintuitively, write operations (``POST /api/inquiries``) executed nearly twice as fast (average 638 ms, median 583 ms) as read operations (``GET /api/farms`` at 1,194 ms and ``GET /api/farms/harvests`` at 1,211 ms). Inspection of the backend codebase reveals that ``getAllFarms`` and ``getHarvestHistory`` execute Mongoose ``populate('farmer')``. In Mongoose, ``populate()`` executes a secondary relational lookup query and performs in-memory JavaScript document stitching in Node.js runtime memory. Under 50 parallel requests, repeatedly building populated document trees creates garbage collection pressure and event-loop lag. In contrast, ``createInquiry`` performs a direct, single MongoDB document insertion.

3. Network Transport & Cross-Cloud Hop Overhead

The minimal response time observed across all endpoints was ~484 ms. This indicates that baseline latency is heavily governed by geographical distance and multi-cloud latency: JMeter Client (Local ISP) -> Render Edge Router (US/Frankfurt) -> Node.js Container -> MongoDB Atlas Cloud Cluster. A minimum baseline of ~400-500 ms round-trip time is typical for cross-regional SSL/TLS handshakes and database connection round-trips without localized edge CDN acceleration.

5. Strategic Engineering Recommendations

Priority Area	Recommendation	Expected Impact
Database Indexing	Add compound index { status: 1, createdAt: -1 } and { farmer: 1 } in farm and harvest schemas to prevent full-collection scans during populate calls.	Reduces query execution time by 60-70%
In-Memory Caching (Redis)	Introduce Redis or Node-Cache with a 60-second TTL on `/api/farms` and `/api/farms/harvests`. Farm records change infrequently; caching will drop read latency from 1.2s to <50ms.	Cuts API read latency from 1,200ms to <50ms
MongoDB Aggregation Pipeline	Replace heavy Mongoose `.populate()` with native `\$lookup` aggregation pipelines and `.lean()` projection queries to eliminate Node.js document stitching overhead.	Reduces Node.js heap and GC pause times
Bcrypt Thread Pool Tuning	Increase Node's libuv worker thread pool (`UV_THREADPOOL_SIZE=8` or `16`) in the Dockerfile, or reduce bcrypt salt rounds from 12 to 10 for standard auth tokens.	Prevents CPU queue bottlenecks during logins
Render Tier Scaling	Upgrade from Render's burstable/shared tier to a Dedicated Instance (1 vCPU, 2 GB RAM) or configure horizontal autoscaling with 2 min-instances to handle concurrent bursts.	Eliminates 18s response spikes on traffic spikes

Report Prepared By: S.H.S. Hansara - Automated Performance Testing Pipeline & SRE Review
AgriConnect Deployment Infrastructure
Status: Certified Production Baseline